

```

        maze[(x, y - 1)] = EMPTY # Connecting hallway.
    elif nextIntersection == SOUTH:
        nextX = x
        nextY = y + 2
        maze[(x, y + 1)] = EMPTY # Connecting hallway.
    elif nextIntersection == WEST:
        nextX = x - 2
        nextY = y
        maze[(x - 1, y)] = EMPTY # Connecting hallway.
    elif nextIntersection == EAST:
        nextX = x + 2
        nextY = y
        maze[(x + 1, y)] = EMPTY # Connecting hallway.

    hasVisited.append((nextX, nextY)) # Mark as visited.
    visit(nextX, nextY) # Recursively visit this space.

# Carve out the paths in the maze data structure:
hasVisited = [(1, 1)] # Start by visiting the top-left corner.
visit(1, 1)

# Display the final resulting maze data structure:
printMaze(maze)

```

Copy this JavaScript code to a file named *mazeGenerator.html*:

JavaScript

```

<script type="text/javascript">

const WIDTH = 39; // Width of the maze (must be odd).
const HEIGHT = 19; // Height of the maze (must be odd).
console.assert(WIDTH % 2 == 1 && WIDTH >= 2);
console.assert(HEIGHT % 2 == 1 && HEIGHT >= 2);

// Use these characters for displaying the maze:
const EMPTY = "&nbsp;";
const MARK = "@";
const WALL = "&#9608;"; // Character 9608 is '■'
const [NORTH, SOUTH, EAST, WEST] = ["n", "s", "e", "w"];

// Create the filled-in maze data structure to start:
let maze = {};
for (let x = 0; x < WIDTH; x++) {

```